

Практическая работа №1

Практическая работа №1. Разработка UDP-протокола и проектирование архитектуры

Общее описание

Цель: заложить основу сетевого взаимодействия для сквозного проекта. Студенты разрабатывают UDP-клиент и сервер с поддержкой базовых игровых команд, а также проектируют архитектуру будущей игры (выбор топологии, распределение обязанностей, спецификация протокола). Вся работа ведётся в GitVerse.

Результат: работающий UDP-обмен с сериализацией игровых структур, спроектированная архитектура, зафиксированная в репозитории документация.

Задание

Разработайте консольное приложение на C++ (или C# – по выбору), состоящее из двух программ:

1. **UDP-сервер** – принимает пакеты от клиентов, обрабатывает команды (движение, выстрел) и отправляет обновлённое состояние.
2. **UDP-клиент** – отправляет команды на сервер и получает ответы.

Требования:

- Использовать системные сокеты (Berkeley Sockets под Linux/Windows или `UdpClient` в C#).
- Реализовать **двусторонний обмен**:
 - Клиент отправляет пакет с заголовком (тип команды, порядковый номер, размер данных) и полезной нагрузкой.
 - Сервер распознаёт тип, обрабатывает данные и возвращает подтверждение или обновлённое состояние (например, новую позицию).
- Поддерживать как минимум **два типа команд**: `MOVEMENT` (передача координат) и `SHOOT` (имитация выстрела).
- Сервер должен вести лог всех полученных команд.

- Клиент должен отправлять команды с заданной периодичностью (например, раз в секунду) и выводить ответ сервера.

Дополнительное задание (для продвинутых):

- Добавить в заголовок поле `Checksum` (контрольная сумма CRC32) для проверки целостности.
- Реализовать простую авторизацию: клиент отправляет логин и пароль, сервер проверяет по заранее заданному списку.

План выполнения работы (на два семинара)

Семинар	Этап	Длительность	Действие
Семинар 1 (1,5 ч)	1	15 мин	Введение и выбор архитектуры: краткий обзор жанров и топологий (Dedicated, Listen, P2P, Relay). Каждая команда (2–3 чел.) определяет свой жанр (FPS, гонка, MOBA) и выбирает топологию, руководствуясь критериями (число игроков, допустимые задержки, бюджет).
	2	45 мин	Разработка UDP-эхо-сервера: создание сокета (<code>socket(AF_INET, SOCK_DGRAM, 0)</code>), привязка к порту (<code>bind()</code>), запуск цикла приёма пакетов (<code>recvfrom()</code>). Сервер отвечает тем же сообщением (<code>sendto()</code>). Разбор структуры <code>sockaddr_in</code> , порядка байт (<code>htons()</code> , <code>htonl()</code>).
	3	30 мин	Разработка UDP-клиента: создание сокета, отправка тестовой строки на сервер, получение ответа. Тестирование на <code>127.0.0.1</code> . Фиксация кода в GitVerse (ветка <code>feature/udp-echo</code>).
Семинар 2 (1,5 ч)	4	20 мин	Проектирование архитектуры: на основе выбранной топологии команды определяют основные классы (игрок, мир, серверный менеджер, клиентский рендерер) и распределение обязанностей (сервер – авторитет, клиент – отображение и ввод).

			Оформляют схему в <code>docs/Architecture_Design.md</code> .
	5	50 мин	Расширение UDP-протокола: добавление заголовка с <code>PacketType</code> (enum), <code>SequenceNumber</code> (uint16_t) и <code>PayloadSize</code> . Реализация сериализации/десериализации структур (позиция <code>float x, y, z</code> , выстрел <code>uint8_t weaponId</code>). Клиент отправляет команду движения, сервер обрабатывает и возвращает подтверждение или обновлённую позицию.
	6	20 мин	Интеграция в GitVerse: создание Pull Request с кодом, описание изменений. Заполнение файла <code>docs/Protocol_Specification.md</code> – спецификация формата пакетов (тип, длина, поля). Внесение в <code>README.md</code> описания проекта и ссылок на документацию.

Критерии оценки

Критерий	Вес	Описание
Работоспособность UDP-обмена	30%	Клиент и сервер успешно обмениваются пакетами, сервер отвечает на команды.
Правильность сериализации	20%	Структуры правильно упаковываются и распаковываются (порядок байт, выравнивание).
Архитектурное проектирование	20%	Обоснован выбор топологии, документирована схема классов и обязанности компонентов.
Документация (спецификация протокола)	15%	Наличие <code>Protocol_Specification.md</code> с описанием всех полей пакетов.
Работа в GitVerse	15%	Ветки, осмысленные коммиты, Pull Request, наличие документации в репозитории.

Сопровождающие ссылки

Основная документация по сокетам

- **Beej's Guide to Network Programming** – классическое руководство, разделы по UDP (Datagram Sockets). Охватывает `socket()`, `bind()`, `sendto()`, `recvfrom()`, порядок байт.
<https://beej.us/guide/bgnet/>
- **Berkeley Sockets – конспект лекций (UW-Madison)** – краткое описание системных вызовов с примерами для UDP.
<https://pages.cs.wisc.edu/~suman/courses/640/s18/sockets.pdf>
- **Berkeley Sockets – справочный материал (Binghamton University)** – таблица вызовов для TCP/UDP.
<https://www.cs.binghamton.edu/~nael/classes/cs528/berkeleysockets.pdf>

Примеры кода

- **UDP-эхо-сервер на C++ (GitHub)** – готовый пример с `bind()`, `recvfrom()`, `sendto()`.
https://github.com/SAIRISAN123/Networking_Template_Code/blob/main/UDP_C%2B%2B/UDP_Server.cpp
- **UDP-клиент на C++ (GitHub)** – пример отправки сообщения.
<https://github.com/naoko-portfolio/tcp-udp-socket-programming-cpp>

Сериализация и структуры

- **Руководство по выравниванию структур** – как упаковывать данные без выравнивания (`#pragma pack`).
<https://en.cppreference.com/w/cpp/language/object>
- **CRC32 – реализация на C++** – пример вычисления контрольной суммы (для доп. задания).
<https://create.stephan-brumme.com/crc32/>